
When Do Language Servers Help Coding Agents?

Ian Barber

July 2026

Abstract

Coding agents are increasingly given language-server tooling, such as go-to-definition, compact definition spans and type diagnostics, on the assumption that better code intelligence makes a better agent. We tested that against a capable text baseline of grep, ranged reads and a shell, using seven models over constructed dispatch and seeded-defect suites, retrieval tasks built on real library source, and a shell agent on real repositories.

Availability alone rarely paid. Go-to-definition was token-neutral wherever the receiver’s type was readable in source the agent already opens, because the model reads the type and localizes itself; it lifted resolution only where the type was withheld, and then only when the agent was prompted to use it. A compact span, produced by a static definition tool rather than a live server, cut total tokens 3.3–4.1x against whole-file retrieval, and 1.3x against grep plus ranged reads in the one suite where it genuinely replaced the read; elsewhere models read the file anyway, on 35 of 36 pushed spans. A type checker lifted accepted correct outcomes from 1/12 to 11/12 at the submission boundary and changed nothing during editing, on a channel the model exercised once in twelve.

What moved outcomes in every thread was a change in the agent’s policy, not the presence of the tool. Thirty-nine demonstrations taught a 27B to stop rereading, and the learned policy proved conditional rather than blanket: where the span genuinely lacked the defect it went looking every time. Installing the server is not the intervention.

1 Introduction

A language server answers questions about code. So does a coding agent holding grep, ranged reads and a shell, and it answers most of them for itself. What the server adds therefore turns on three narrower questions: whether it answers something the agent cannot, whether it answers in a cheaper form, and whether it answers at a better moment. Prior work on each rarely reports that counterfactual, so it is the baseline used throughout.

IF: whether delivering the semantic information changes what the agent finds. Typed Holes [2] and LSPRAG [3] push types and definitions into the context on the premise that the information is missing, which is the premise this question tests. A yes is the semantic arm resolving targets the text arm misses; a no is both arms resolving the same tasks at the same cost.

FORM: whether a compact span costs less than reading the file. CodeStruct [4] makes the neighbouring case that structured reads and edits beat whole-file handling by exposing program structure. A yes is total tokens falling at unchanged success; a no is the span arriving as context on top of the reads it was meant to replace.

WHEN: whether the moment a diagnostic arrives changes the outcome. STALL+ [5] finds static-analysis value varying with integration phase, and CoCoGen [1] checks a coherent draft before retrieving context to repair it. A yes is the same diagnostic changing the accepted result depending only on when it fires; a no is the outcome flat across delivery phases.

2 Method

We evaluated in two settings. The first is a purpose-built harness in which a model drives a fixed loop of six actions over a Python workspace: `grep`, ranged read, whole-file read, line edit, run tests, and submit. Each arm adds or withholds exactly one semantic operation and holds the rest constant, which is what makes the comparisons causal. The second is an off-the-shelf shell agent, `mini-swe-agent` driving Claude Sonnet 4.5 with ordinary shell primitives, run on three SWE-bench tasks in `sympy`, `astropy` and `sphinx` with an AST-backed definition command available, one seed and a 60-step cap. That setting sacrifices control for realism and we treat it as a case study throughout. Nothing was tested through an MCP server, a skill or an editor plugin, so these experiments can say which operation helps but cannot rank delivery surfaces against each other.

We created four task suites. The dispatch suite has fifteen synthetic tasks in which a typed receiver calls one of roughly ten identically named overrides, exactly one of which is buggy. It runs across a ladder that moves the receiver’s type progressively further from the call site, from a call-site annotation to the test’s construction site to factory indirection, and a fourth rung that hides the type altogether by withholding the application and test source and redacting the receiver construction. Two suites measure retrieval cost: one is fully synthetic, and the other builds constructed misuse tasks over vendored library source from `toolz` 1.1.0 and `more-itertools` 11.1.0. A fourth suite supplies mechanically validated definition-span instances for the substitution experiments. The checker suite has twelve pairs of seeded defects and clean controls, where each defect is coherent, passes the visible test, fails a held-out test, and carries exactly one semantic diagnostic in the target file, and each clean control is that defect’s own validated gold. We built tasks rather than harvesting them because a bounded scan of real repositories produced no candidate that cleared environment, leakage, ambiguity and discriminating-lookup checks together.

The definition operation in the retrieval experiments is a static AST resolver rather than a live language server. Live Pyrefly served the dispatch lookup arms and one 7B run.

The dispatch ladder, the definition-span instances, the paired retrieval comparison and the checker grid ran at temperature 0 with a single rollout per cell. The hidden rung and its matched visible control ran at temperature 0.7 over five seeds per task, 450 rollouts in all. Temperature 0.7 with repeated seeds also covers the whole-file contrast, the election and execution-feedback runs, and the training harvest, whose held-out retest ran at temperature 0. We ran Qwen2.5-Coder-7B, Qwen3.5-27B and Qwen3.6-27B locally, and Claude Sonnet 4.5, DeepSeek v3.1, GLM-4.6 and GPT-5.6 “Luna” through an API.

We report five measures. Resolution is whether the agent fixed the intended target and passed a held-out test written against the specification rather than the visible one. Cost is total tokens, input plus output, over the whole trajectory. Election counts how often the agent chose a semantic operation when one was offered. Substitution counts reads of the defining file after a definition span had already been delivered, which is the behaviour that decides whether a span saves anything. For the checker experiments we also report wall time, since a diagnostic that prevents a bad submission trades latency for a repair. The unit of analysis is the task, and intervals are task-level bootstraps.

Two limits follow from this design. The workspaces are small enough that an agent can read any file it wants under budget, so the results speak to the regime where reading is cheap and not to codebases where it is not. And static tooling can only reach what a type checker sees, which leaves out dynamic behaviour, wrong annotations and `Any`-heavy boundaries. All code, task definitions and run artifacts are in the repository accompanying this report.

3 Results

3.1 Does delivering the semantic information change what the agent finds?

Where the receiver’s type was visible in the source, semantic lookup made no difference. Text search resolved all fifteen dispatch tasks, and go-to-definition resolved fourteen unprompted and fifteen when the agent was told to use it, at matched-success token ratios of 0.972 and 1.041. Moving the type further from the call site did not change that. Mean input tokens on resolved tasks were 1,436, 1,429 and 1,465 across the three ladder rungs, resolution stayed at fourteen or fifteen, and lookup cost between 0.945 and 1.065 times text search at every rung, including the factory-indirection rung where only a type-aware server can resolve the receiver statically. The trajectories explain the flatness: the agent grepped about 0.3 times per task, read the receiver’s type wherever it happened to appear, and opened the buggy override first.

Hiding the type changed the answer. The fourth rung withholds the source that carries the type, so the agent has to retrieve it rather than read it, and we ran it at five seeds per task against a matched visible control:

Arm	Hidden	Paired difference vs grep	Grep	Whole-file reads	Visible
grep_base	70/75	—	1.52	2.17	75/75
defn_avail	71/75	+0.013 [−0.040, +0.067]	1.47	2.04	75/75
defn_prompt	75/75	+0.067 [+0.013, +0.133]	0.36	0.68	75/75

Per-task resolution rates over five seeds at temperature 0.7, 95% task bootstrap intervals over the fifteen tasks; every arm resolved every rollout in the visible control, so the effect appears only where the type was withheld.

Prompting the agent to use lookup improved four tasks and worsened none. Offering the same operation without prompting improved three and worsened two, which leaves it indistinguishable from text search. Every arm solved every rollout in the visible control, so the paired difference there is exactly zero and the effect exists only where the type was withheld. The cost of hiding the type fell on resolution rather than on tokens: among tasks both arms solved, lookup cost 0.983 times text search when prompted and 1.005 when merely available. This rung is constructed, and it withholds source a real agent could usually open.

A separate pilot tested the resolver rather than the agent, on repositories that are byte-identical apart from one stub. Typed lookup reached the exact intended override among eight to fifteen identically named candidates, where the erased variant returned a base result that cannot discriminate between them, and both variants type-check cleanly. That precision bought nothing at the agent level. All twelve task-condition cells passed, the typed automatic arm cost 1.037 times the textual arm with an interval from 0.988 to 1.093 that is too wide to call equivalence, every automatic result was still followed by a read of the target file, and composing the lookup added about six seconds per task.

3.2 Does a compact span cost less than reading the file?

Against a whole-file read the span is much cheaper. It cut total tokens by 3.35x for Qwen3.6-27B, 3.49x for Sonnet 4.5 and 4.12x for DeepSeek v3.1, and no attempt failed under either interface for any model.

That comparison flatters the span, because a capable agent rarely reads a whole file. The harder test gives the text arm grep, ranged reads and a whole-file fallback. On eleven tasks over real library source, Qwen3.5-27B spent 1,602 mean total tokens with text retrieval and 1,235 with definitions, a paired ratio of 1.297 with a bootstrap interval from 1.093 to 1.527, and definitions were cheaper on ten of the eleven. Both arms solved everything. This is also the one suite and model where the span genuinely replaced the read, with none of the eleven results followed by a reread, and that turns out to be the exception rather than the rule.

The saving therefore depends on two agent behaviours. The first is election, choosing the operation at all, and it is the tractable one. An untrained 7B invoked the definition operation in 1 of 48 rollouts; after relabel training it invoked it in all 48, stopped reading files entirely, and moved from 3,086 to 687 mean input tokens while resolution rose from 31 to 48. Prompt framing raised use of the equivalent command on two of three tasks in the shell-agent case study. A second pair of runs isolates policy from backend: with the tool enabled in both arms, the untrained 7B never invoked it across 24 rollouts and the trained model invoked it every time, moving mean input from 2,894 to 689 tokens and success from 14 to 24. A live Pyrefly backend reproduces the saving once the policy elects, but no arm held the policy fixed while varying the backend, so this measures the policy and not the server.

The second behaviour is substitution, actually using the span instead of the file, and it fails almost everywhere. When the span was pushed into the context unsolicited, models opened the defining file anyway on 35 of 36 instances. Telling them explicitly that the span was complete and the file should not be opened removed two of those and cost the 27B 294 extra tokens per task.

Model	Reread, pushed span	Reread, plus sufficiency instruction
Qwen3.6-27B	11/12	12/12
Sonnet 4.5	12/12	12/12
DeepSeek v3.1	12/12	10/12

Letting the model ask for the span rather than pushing it does not fix this. A separate run with Qwen3.6-27B, GLM-4.6 and GPT-5.6 “Luna” reproduced the pushed reread at 35 of 36, and among the instances each model chose to request the span, it still reread on 9 of 9, 4 of 9 and 7 of 12. GLM-4.6 solved only five or six of these twelve tasks, and the reread was counted independently of whether the task succeeded.

The behaviour is not an artifact of the constructed apparatus. In the shell-agent case study, roughly 16 of 18 definition calls were followed by a read of the file just resolved, in one case 22 ranged reads of the same sympy module. The same agent took between 0 and 3 whole-file reads in 44 to 60 actions, which is why the large whole-file ratios above do not describe its situation.

Training is the only lever that worked. A DAgger-style relabel run [6] harvested 39 demonstrations, none of which required a read, and cut the reread from 11 of 12 to none on held-out instances built from different seeds and different templates, moving mean input from 1,157 to 748 tokens for a saving of 1.59x on tasks both arms solved. Held-out pass moved from 11 to 10 of 12, one rescue against two losses that pass the visible test and fail the specification, which at twelve instances and one seed is not distinguishable from noise. We then spent a reserved set of twelve instances, disjoint from the harvest in both seeds and templates and never run before, as a pre-registered confirmation. It reproduced the effect: the reread fell from 12 of 12 to 1 of 12, with eleven removed, one persisting and none induced, mean input fell from 1,223 to 729 tokens for a saving of 1.725x, and held-out pass moved from 12 to 11.

Because every training example had a span that already contained the defect, this leaves open whether the model learned to judge sufficiency or simply to stop reading. To separate those, we built twelve instances that hoist each override’s arithmetic into a module-level helper, so the span the server returns for the call site is still the complete definition of the method that binds but the defect now sits outside it. A twin repository differing in exactly one line restores sufficiency, and a third arm leaves the repository byte-identical while delivering a second genuine lookup at the helper.

Reads after span	Span insufficient	Span sufficient	Helper also delivered
Untrained	12/12	12/12	11/12
Trained	12/12	2/12	0/12

The trained model went looking every time the span lacked the defect, and largely stopped when it did not. The untrained model read in all three situations, so it never made the distinction at all. Both conditions repaired every instance in every arm, on the visible and the held-out test, which means the manipulation moved retrieval behaviour without moving outcomes. Where reading was necessary the trained model read once on average against the untrained model’s 3.42 times, and where it was unnecessary, 0.17 and 0.00 times against 1.58 and 1.75. Input tokens were lower in every arm. Two apparatus checks matter for reading this result. An adversary over seventeen one-parameter repair forms found none that reached the held-out test without retrieval, so the instances cannot be solved by guessing. The harness re-shows any file the agent has edited, which would have delivered the hidden line without a read, so that view is redacted while deliberate reads and grep remain available.

3.3 Does the moment the diagnostic arrives change the outcome?

We compared three delivery points against a no-checker control: a diagnostic after every edit, a single diagnostic at revision, and a gate at submission that rejects a defective draft and asks for a repair. Accepted submissions that were both type-clean and correct against the held-out test rose from 1 of 12 with no checker to 10 of 12 at revision and 11 of 12 behind the gate, effects of +0.375 [+0.250, +0.500] and +0.417 [+0.292, +0.500]. Delivery after every edit stayed at 1 of 12, an effect of +0.000 [−0.125, +0.125], but that arm hardly ran: the model edited before submitting on only 1 of 12 defects, so the checker fired once. It is unexercised rather than refuted.

Twelve seeded defect/clean pairs	Control	After every edit	One-shot	Gate
Bad completion accepted	11/12	11/12	2/12	1/12
Revision tokens, defect	787	754	1,210	1,380
Wall time, defect (s)	27.8	14.5	68.3	62.7
Revision tokens, clean	591	—	619	591
Wall time, clean (s)	9.4	9.9	9.6	9.5

After-every-edit fired in 1 of 12 defect rows; mean edits 0.08 against 0.17 in control.

Cost tracks how often the checker speaks. On the clean drafts, which needed no diagnostic at all, the after-every-edit arm cost 651 tokens and the revision arm 619, against 591 for both the control and the gate. No arm produced a false rejection and all twelve clean drafts were accepted everywhere, so this is unconditional cost rather than spurious complaint: the two earlier arms hand over a diagnostic whether or not the draft needs one, while the gate speaks only when the submission is actually defective. On clean work the gate therefore matched control in both tokens and wall time, and it spent about 2.2 times control’s wall time on defective work. It rejected 10 of 12 submissions, and every rejection ran the full repair, retest and resubmit cycle; the remaining two defects were repaired before the model submitted. Its one-task advantage over revision delivery came from refusal rather than information: on one task the model received the diagnostic, made no edits, and submitted anyway, and only the gate’s rejection forced a repair. Its single miss is instructive in the other direction, since the model self-repaired into a state that was type-clean and still failed the held-out test, which is the limit of any gate built on a type checker.

Two caveats bound this. The zero false rejections on clean drafts are an apparatus check rather than a measurement, because each clean control is that defect’s own validated gold and the checker is deterministic, so it could not have rejected them; the rate on real work is unmeasured. And the defects had to be seeded because capable models rarely leave checker-detectable ones: frontier inference sat at ceiling, no natural 7B draft was coherent enough to use, and the coherent 14B drafts were already type-clean. A separate grid on execution feedback, covering two frontier models, fourteen tasks, three seeds and three delivery modes, passed every one of its 252 attempts.

4 Discussion

4.1 Delivery

In most cases semantic navigation added little over `grep` and ranged reads. A capable model reads the receiver’s type wherever it appears in the code it already has open, works out which override binds, and goes to the right file on its own, so the lookup returns something the agent could already derive. Moving the type further from the call site did not change that. What carries resolution is the type being present and correct in the source, not the server that queries it.

Navigation helps when the type is not in the code the agent can see: when it lives in a generated stub, behind a compiled extension, inside a vendored package, or on an object constructed somewhere the agent has not opened. The agent then has to fetch the type rather than read it, and lookup resolved tasks that text search could not. Even there, making the operation available was not enough. The agent had to be told to use it, and the arm that merely offered it performed like text search. Sound types also sharpen the resolver itself, distinguishing the right override among many identically named ones, but that precision produced no gain at the agent level.

4.2 Form

A compact span is cheaper than a read only when it replaces the read, and whether it does is a property of the agent’s policy rather than of the tool. Against a whole-file read the span wins by a wide margin. Against competent `grep` and ranged reads the margin is narrow, and that is the margin a real coding agent is working with, because it rarely reads a whole file in the first place.

Two behaviours sit between the operation and the saving, and they proved very different. Election, choosing the operation at all, moves easily: prompt framing shifted it on capable models and training shifted it on a small one. Substitution, using the span instead of opening the file, resisted everything we tried. Models reread the file when the span was pushed at them, when they were told it was complete and not to open the file, and when they had asked for it themselves. The same behaviour appeared in a shell agent working on real repositories, so it is not a quirk of the constructed tasks.

Training moved substitution where nothing else did, and what it produced is a judgment rather than a reflex. Given a span that was genuine but did not contain the defect, the trained model went looking every time, and went once where the untrained model went three times. Given a span that sufficed it mostly stopped, and given the missing definition as well it stopped entirely. Correctness did not change in any of these conditions, so what training bought is an agent that retrieves when retrieval is warranted and not otherwise. It learned that distinction from demonstrations in which reading was never the right move.

4.3 Timing

With the type checkers we have today, the end of the turn is the right place to run one. At the submission boundary the checker turned accepted bad completions into repaired ones, and a single diagnostic at revision did nearly as well. Delivering the same checker after every edit changed nothing, though the model so rarely edited before submitting that the channel barely ran, so that arm is unexercised rather than refuted.

Of the two late options the gate is the better deal, because it charges only defective work. Diagnostics at revision arrive whether or not the draft needs them, so every clean draft pays; the gate speaks only when a submission is actually broken, and clean work costs exactly what it did without a checker. Its ceiling is the checker behind it, since a submission can clear a type check and still be wrong, which is what happened on the one defect the gate let through. The opportunity is scarce in any case, because capable models rarely leave a checker-detectable defect at all.

That conclusion is about today's checkers rather than about timing in principle. A type checker is built to report on every change because it was designed for a human in an editor, so an agent can only choose between taking all of its output or taking it at one chosen moment. A checker designed for an agent could instead decide when a diagnostic is worth interrupting for, and the interesting question is not when to run the checker but whether the checker can learn when to speak.

5 If you use a coding agent

Run your type checker at the end of the agent's turn and make it blocking. This is the clearest return in the report and the cheapest to wire, because most harnesses expose a hook at exactly that point. It costs nothing when the agent's work is already clean, since a passing check produces no output and no repair. To see whether it is earning anything on your codebase, count how often it blocks and how often the resulting repair passes.

Do not add a navigation tool if your types are already written where the agent reads. The test is one you can run by eye: open a file your agent works in and ask whether you could tell what type a variable holds from what is on the screen. If you can, so can the model, and it will find the right definition without help.

Consider one where you cannot. The cases that matter are the ones where you personally would have to jump elsewhere to answer that question: types that live only in a generated stub, behind a compiled extension, inside a vendored package, or on an object assembled somewhere else. Installing the tool is not sufficient on its own. In our tests the arm that merely made the operation available performed like plain text search, and only the arm that told the agent to use it improved anything, so put that instruction in your project instructions or the tool description.

Do not assume a definition tool saves tokens until you have checked that the agent stops reading. This is the single most reliable way to be disappointed. Count reads of the defining file that happen after the tool has already returned it; if that number is not near zero, the tool is adding context rather than replacing it, and your token bill will go up.

Fine-tuning works, but only if you own the weights. That rules it out for a hosted agent. Where it is available it buys a genuine judgment rather than a blunt rule, since the trained model still went looking every time what it had been handed was insufficient.

Keep type annotations correct and present; the text baseline exploited exactly that. On your own workload, measure wrong-file edits, post-span reads of the defining file and spurious gate rejections; the clean-work

cost here came from workspaces guaranteed clean. No result here ranks MCP servers, skill files and project instructions as delivery surfaces.

6 Future work

The line most worth following is retrieval calibration. Thirty-nine demonstrations, none of which required a read, produced a model that goes looking when what it holds is inadequate and stops when it is not. That is a judgment about the sufficiency of retrieved context, learned from examples that never exercised it. Nothing in that is specific to definition spans. Agents retrieve constantly and mostly indiscriminately: grep hits, file chunks, documentation, search results, the output of other tools. If “do I have enough?” is trainable rather than a property of one tool, it is a general lever on agent cost, and the open questions are general ones. Does the judgment transfer across retrieval channels, or was it learned about one? Does it survive forms of inadequacy structurally unlike the delegation tested here, or is the model reading a surface cue rather than testing sufficiency? Does it hold at other scales, outside a task-specific adapter, and on a real codebase, where sufficiency is a matter of degree rather than a constructed fact?

The second line is co-designing the tool with the agent. Every tool tested here was built for a human in an editor. A type checker reports on every change because a person can glance at a squiggle and ignore it, and a language server answers the question it was asked because a person chose to ask. An agent inherits both defaults and can only take all of the output or take it at one moment it picks, which is why the best available option was to run the checker once at the end of the turn. A checker built for an agent could decide when a diagnostic is worth interrupting for, suppressing what the agent is already about to fix and speaking up when it is heading somewhere expensive; a server built for an agent could return what the caller will need next rather than what it literally asked for. The question is not only when the agent should consult the tool, but whether the tool can learn when to speak. Nothing here tests that, and the gate result is the ceiling of what the human-facing version can do.

The third question is whether a codebase can be assessed in advance. Our results say the discriminator is whether the type is readable in the source the agent already opens, which is a property of the repository rather than of the agent, and yet a practitioner has no way to compute it. Something like annotation density, the depth of indirection between a call site and the definition that binds it, or the share of types that resolve only through a stub or a compiled boundary might predict how much a language server buys before anyone installs one. That would turn a per-workload measurement into a decision that can be read off the repository, and it is the difference between advice and a tool.

The fourth is which tasks benefit most, and which are actively harmed. Everything here is dispatch ambiguity and small seeded defects in workspaces small enough to read. The intuition worth testing is that semantic tooling matters in proportion to how much relevant context sits outside what the agent can afford to read, which would make wide refactors, cross-module renames and API migrations the interesting cases rather than the localised bug fixes studied here. The opposite end deserves attention too: an agent that stops reading is cheaper until the day the thing it needed was in the part it skipped, and we do not know what class of task that failure lands on.

7 Conclusion

Language servers help coding agents when two conditions hold together: the server supplies something the agent cannot cheaply work out for itself, and the agent’s behaviour actually changes to use it. Most of what we tested failed one or the other.

Go-to-definition usually fails the first. The agent reads the types it needs in the code already in front of it and finds the right definition unaided. Where those types are not in that code it does help, but only when the agent is told to use it rather than merely given it. A compact definition span fails the second. It costs less than reading the file only when it replaces that read, and models handed a span opened the file anyway; prompting did not change that and training did. A type check at the moment of submission satisfies both conditions, which is why it was the clearest result here. It catches what the model cannot see in its own work, and a blocking gate compels the change in behaviour rather than inviting it.

The tool was never the intervention. What moved the outcome was a change in what the agent did with it.

References

- [1] Zhangqian Bi, Yao Wan, Zheng Wang, Hongyu Zhang, Batu Guan, Fangxin Lu, Zili Zhang, Yulei Sui, Hai Jin, and Xuanhua Shi. Iterative refinement of project-level code context for precise code generation

- with compiler feedback. In Findings of the Association for Computational Linguistics, 2024. CoCoGen; arXiv:2403.16792.
- [2] Andrew Blinn, Xiang Li, June Hyung Kim, and Cyrus Omar. Statically contextualizing large language models with typed holes. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA2), 2024. arXiv:2409.00921.
 - [3] Gwihwan Go, Quan Zhang, Chijin Zhou, Zhao Wei, and Yu Jiang. LSPRAG: Lsp-guided rag for language-agnostic real-time unit test generation. *arXiv preprint arXiv:2510.22210*, 2025.
 - [4] Myeongsoo Kim, Joe Hsu, Dingmin Wang, Shweta Garg, Varun Kumar, and Murali Krishna Ramanathan. CODESTRUCT: Code agents over structured action spaces. *arXiv preprint arXiv:2604.05407*, 2026.
 - [5] Junwei Liu, Yixuan Chen, Mingwei Liu, Xin Peng, and Yiling Lou. STALL+: Boosting llm-based repository-level code completion with static analysis. *arXiv preprint arXiv:2406.10018*, 2024.
 - [6] Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011. PMLR v15:627–635.